

GNN-DynPen: Complete Project Explanation & Presenter Notes

A. PREVIOUS WORK / LITERATURE REVIEW

What problem were researchers trying to solve before you?

Penetration testing (PT) is the process of ethically hacking your own network to find weaknesses before real attackers do. In large organisations this is done manually by expensive security experts, and it needs to be repeated every time the network changes. Researchers have been trying to automate this using **Reinforcement Learning (RL)** — training an AI agent that learns to attack a network through trial and error, just like how AlphaGo learned to play chess.

The research timeline — what came before

2019 — NASim (Schwartz & Kurniawati) The starting point. They built a Python simulator called NASim (Network Attack Simulator) that models a computer network as a grid of hosts, each with vulnerabilities, operating systems, and services. They showed that a basic Deep Q-Network (DQN) — a type of neural network that learns Q-values (expected future rewards for each action) — could learn to find attack paths in small 5-host networks. This was proof of concept but only worked on tiny, completely static networks.

2020 — Hu, Beuran & Tan Extended DQN to work with attack trees — structured graphs that represent possible sequences of exploit steps. Still static topology. Showed that RL can handle slightly more complex scenarios.

2021 — Tran et al. (Hierarchical RL) Introduced hierarchical agents: a high-level agent picks a goal (e.g. "compromise subnet 2") and a low-level agent picks individual actions to achieve it. This decomposed the action space and helped with larger networks. Still static.

2023 — Chen et al. (GAIL-PT) Used Generative Adversarial Imitation Learning — showed the agent expert human demonstrations to speed up convergence. Novel combination but still tied to a fixed topology.

2023 — Ghanem & Chen (Hierarchical RL for large networks) Applied hierarchical RL to networks with more hosts. Made progress on scalability but still needed full retraining for each new network.

2024 — Li et al. (DynPen) — Your Direct Baseline This is the most important paper. DynPen was the first serious attempt to handle networks that CHANGE at runtime. Their key idea: wrap a pre-trained RL agent in three coordination modules:

1. **Trajectory Logger** — records every (state, action, time) triple during the attack, creating a replayable history of the attack path

2. **Monitoring Module** — periodically rescans the most important nodes (those with highest network centrality — the "busiest" nodes) to cheaply detect when something has changed
3. **Backtracking Module** — when a change is detected, walks backward through the trajectory log to find the most recent step that is still valid in the new topology, then resumes from there

DynPen showed impressive results — outperforming a vanilla RL agent by 10x to 270x under various dynamic regimes. This was a major advance.

BUT — DynPen had one big remaining problem:

The neural network inside DynPen has a FIXED input size. It takes a flat matrix of shape $(N_{\text{max}} \times F)$ — where N_{max} is the maximum number of hosts you decided at design time, and F is the features per host. This means:

- If you train on 5 hosts, you cannot use the policy on a 10-host network
- If a new host is added to the real network, the whole architecture breaks
- Every new network topology = full retrain from scratch
- Action space is all (source, target, exploit) triples — grows as $O(N^2 \times V)$, which explodes

B. THE GAP YOU IDENTIFIED AND FILLED

The precise gap

DynPen solved the "when to adapt" problem (detect a change, roll back, replan). But it did not solve the "how to represent state so the agent works on any network" problem.

More precisely, DynPen's architecture is **topology-specific**:

- Input size hardcoded to $N_{\text{max}} \times F$
- Output size hardcoded to all pre-enumerated action triples
- Moving the trained weights to a different-sized network is mathematically impossible without architectural changes

Your contribution is to solve this at the **representation level** — replace the flat matrix with a data structure (a graph) and an encoder (a GNN) that is inherently size-independent.

Why this matters

In the real world:

- Enterprise networks have hundreds of hosts, not 5
- They change constantly — patches, new servers, decommissions
- Nobody can afford to retrain an RL agent every time a server is added
- You want one trained policy that just works, regardless of network size

The key insight

Graph Neural Networks (GNNs) were designed to process graphs of any size using the SAME learned weights. A GNN trained on 5 nodes uses identical weight matrices on 50 nodes — the weights are shared across all nodes because they encode a function of "how to aggregate your neighbours' information", not "what specific node am I at position index 7".

This property — called **inductive learning** — is exactly what's needed for a topology-agnostic PT agent.

C. WHAT YOU DID — METHODOLOGY

The overall approach

You built **GNN-DynPen**: everything DynPen does, but with the fixed flat-matrix state replaced by a GNN encoder. You also implemented DynPen's adaptation modules as an optional add-on (for RQ4).

Step 1 — Formalising the problem as an MDP

You modeled penetration testing as a Markov Decision Process (MDP):

- **State S**: A dynamic graph $G_t = (V_t, E_t, X_t)$ where:
 - V_t = set of host nodes (can change over time)
 - E_t = network connections between hosts (can change)
 - X_t = per-host feature matrix (OS type, services, vulnerabilities, privilege level, compromise flag)
- **Actions A(G_t)**: A variable-length set of (target_node, exploit_template) pairs. This is the key architectural difference — instead of enumerating all (source, target, exploit) triples at design time ($O(N^2 \times V)$), you only score (target, exploit_type) pairs ($O(N \times K)$) using the node embeddings.
- **Reward R**: Sparse positive reward for compromising the goal host, small negative cost per step (to encourage efficiency), large negative for hitting a honeypot.
- **Transition T**: Stochastic — exploits succeed with a probability defined in the scenario, not always.

Step 2 — The GNN Encoder (GraphSAGE)

You chose GraphSAGE (Hamilton et al., NeurIPS 2017) as your backbone. Here is exactly what it does, step by step:

Input: A graph with 5 nodes (in training). Each node has a feature vector of dimension $F=12$ (OS type, services, exploits, privilege level, compromise status, sensitivity).

Layer 0 (Input Projection): $\text{Linear}(12 \rightarrow 64) + \text{ReLU}$ — maps raw features to 64-dimensional vectors. All 5 nodes go through the same linear layer.

Layer 1 (GraphSAGE aggregation): For each node v , compute:

- $\text{AGG}_v = \text{average of all neighbour feature vectors (mean pooling over neighbourhood)}$
- $h^1_v = \text{LayerNorm}(W^1 \cdot \text{CONCAT}(h^0_v, \text{AGG}_v))$

After this, each node knows about its 1-hop neighbourhood (direct connections).

Layer 2 (GraphSAGE aggregation again): Same operation repeated — now each node knows about its 2-hop neighbourhood (neighbours of neighbours). This lets each node "see" 2 steps away in the network.

Global Readout: $h_G = \text{mean of all node embeddings} = \text{graph-level summary vector (64-dimensional)}$.

Q-Value Head (Action Scoring): For each candidate action (target node v , exploit type e): $Q(v, e) = \text{MLP}(h_v \parallel h_G \parallel \text{embed}(e))$

This concatenates the target node's embedding, the whole-graph context, and a learned embedding for the exploit type, then passes through a small MLP to get one scalar Q-value.

Why this is size-invariant: The weight matrices W^1 , W^2 , and the MLP weights do not depend on N . Whether there are 5 nodes or 50 nodes, the same matrices are applied. The number of Q-values computed equals the number of (node, exploit) pairs in the current graph — automatically variable.

Step 3 — The Baseline Agent (Reproducing DynPen)

To have a fair comparison, you reproduced DynPen's input scheme exactly:

- Take the observation matrix $X \in \mathbb{R}^{(N \times F)}$
- Zero-pad it to $\mathbb{R}^{(N_{\text{max}} \times F)}$ ($N_{\text{max}} = 5$ for the 5-host training network)
- Flatten to a vector of dimension $N_{\text{max}} \times F = 60$
- Pass through a 3-hidden-layer MLP (256 units each) to get Q-values for all pre-enumerated actions

This is the hard-coded fixed approach that DynPen uses. On a 10-host network, it literally breaks — you cannot feed $10 \times 12 = 120$ features into a network expecting 60.

Step 4 — Training Protocol (Double DQN)

Both agents use the same training algorithm for fairness:

Double DQN (van Hasselt et al., AAAI 2016) — an improved version of DQN that reduces overestimation of Q-values:

- One "online" network selects the best action from the next state
- One "target" network evaluates how good that action actually is
- Using two separate networks makes training more stable

Epsilon-greedy exploration:

- Start with $\epsilon=1.0$ (take completely random actions 100% of the time)
- Linearly decay to $\epsilon=0.05$ over 50,000 steps
- This means the agent explores randomly at first, then gradually exploits what it has learned

Replay buffer:

- Store transitions (state, action, reward, next_state, done)
- Sample random mini-batches during training — breaks temporal correlations
- GNN version uses 20,000 transitions (smaller due to graph storage overhead)

Key hyperparameters:

- Learning rate: $1e-3$ (Adam optimizer)
- Discount factor $\gamma = 0.99$ (nearly equal weight to future rewards)
- Training: 100,000 steps per seed, 3-5 seeds, report mean $\pm$ std
- Target network update: every 1,000 gradient steps
- Gradient clipping: global norm ≤ 10.0 (prevents exploding gradients)

Step 5 — The Four Experiments

RQ1 — Convergence on fixed 5-host topology: Train both agents from scratch on the same fixed network. Plot cumulative reward vs training steps. Check: does GNN get at least 90% of baseline reward? Answer: Yes — GNN gets 82.4 vs baseline 75.3. GNN actually converges slightly FASTER (reaches 80% of final performance at step 35k vs step 45k for baseline) because the graph structure provides richer inductive biases.

RQ2 — Zero-shot generalisation: Take the trained GNN (weights frozen, no fine-tuning). Evaluate on three topologies it has never seen: 3-host (smaller), 8-host (larger), 10-host (largest). The baseline cannot be evaluated on these at all — it breaks architecturally.

Results:

- N=3: 65.2 ± 8.2 (works on smaller networks)
- N=5: 82.4 ± 4.1 (training topology, both agents equal)
- N=8: 57.3 ± 10.5 (69.5% of training performance — above 50% criterion)
- N=10: 49.8 ± 12.3 (60.4% of training performance — above 50% criterion)

RQ3 — Dynamic topology changes: Train GNN and baseline on fixed 5-host network. Then at runtime, inject changes:

- Property change: patch a vulnerability (change host features)
- Connectivity change: add or remove a network edge
- Node count change: add a new host (baseline BREAKS here — cannot handle)
- All random: mix of all above

Measure: reward AFTER the change starts.

- GNN post-change: 58.3–68.1 (retains 85-90% of pre-change performance)
- Baseline post-change: 12.9–18.4 (collapses to under 25%)
- Improvement: +270% to +352% better than baseline

RQ4 — Backtracking ablation: Under the connectivity change regime, compare three conditions:

- GNN alone: 64.7 ± 7.2
- GNN + backtracking module: 71.8 ± 5.1 (+10.9% absolute gain)

This shows GNN encoding and backtracking are COMPLEMENTARY — they solve different problems and both help.

D. YOUR RESULTS IN PLAIN ENGLISH

RQ	What you asked	What you found	Practical meaning
RQ1	Can GNN learn as well as baseline on a fixed network?	Yes — GNN gets 109% of baseline reward	No cost to switching to GNN
RQ2	Can one trained GNN work on networks it never saw?	Yes — 69.5% and 60.4% on 8 and 10-host networks	Deploy once, works everywhere
RQ3	Does GNN survive topology changes better?	270–352% better than baseline	No retraining needed after changes
RQ4	Does adding backtracking help further?	+10.9% additional reward	Both mechanisms are complementary

E. FUTURE WORK — WHY AND HOW

1. Curriculum training over topology distributions Currently trained only on $N=5$. Training on a mixture of $N=\{3,5,8,10,15\}$ simultaneously (curriculum learning) would improve zero-shot performance on large graphs. Predicted improvement: close the 69.5% gap at $N=8$ to above 85%.

2. Attention-based global pooling (Set2Set, PNA) Currently using plain mean-pooling for graph-level summary. Attention pooling would let the GNN focus on the most important nodes (e.g. the gateway node) rather than averaging all equally. More expressive, especially for asymmetric topologies.

3. 3-layer GNN for larger graphs 2-layer GNN sees 2-hop neighbourhood. For a 10-host sparse network, 2 hops may not reach the goal host. 3 layers = 3-hop context = longer path awareness.

4. PPO / Actor-Critic DQN is off-policy (uses a replay buffer). PPO (Proximal Policy Optimization) is on-policy but generally more stable for complex action spaces and continuous rewards. Worth comparing.

5. Multi-agent (blue team vs red team) Add a defender agent that can patch vulnerabilities and add firewall rules. Train both simultaneously as a zero-sum game (GAN-style). More realistic and would lead to more robust attack policies.

6. Real vulnerability data (CVE embeddings) Currently using synthetic vulnerability categories. Replace with embeddings of real CVE descriptions (Common Vulnerabilities and Exposures database) using a pre-trained text encoder. Would make the simulator much closer to real-world PT.

7. GNN Explainability Apply GNNExplainer (Ying et al., 2019) to identify WHICH edges and node features the agent relies on most when making exploit decisions. This is important for security auditors who want to understand WHY the agent chose a particular attack path.

8. Scale to CyberBattleSim / NASim Validate on the Microsoft CyberBattleSim environment and the standard NASim benchmark to allow direct quantitative comparison with published results.

PART 2 — SLIDE-BY-SLIDE PRESENTER NOTES

SLIDE 1 — Title Slide

What to say:

"Good morning/afternoon everyone. My name is [your name] and today I'm presenting my MS research project titled GNN-DynPen — which stands for Graph Neural Network Dynamic Penetration Testing.

This project sits at the intersection of two fields: cybersecurity automation and machine learning. Specifically, I've been working on automated penetration testing — building an AI agent that can find attack paths in computer networks — and I've extended it to work across networks of different sizes, which previous work could not do.

This project builds directly on a 2024 IEEE paper by Li et al. called DynPen, which I'll explain shortly. I used a simulator called NASim — Network Attack Simulator — to run all my experiments.

Let me walk you through what I've done, why it matters, and what I found."

Time: ~45 seconds

SLIDE 2 — Introduction & Motivation

What to say:

"Let me start with the motivation — why does this problem matter?

On the left, you can see the problem. Modern enterprise networks are NOT static. Every day, IT teams add new servers, patch vulnerabilities, change firewall rules, remove old machines. This makes the network a constantly moving target.

Existing AI-based penetration testing agents — including the best one before my project, called DynPen — were designed with a fundamental assumption: they expect the network to be a fixed, pre-specified size. The neural network inside them has an input layer of a fixed size, say 5 hosts times 12 features = 60 inputs. If the network now has 6 hosts, the architecture literally breaks. You have to retrain the entire agent from scratch.

On the right is my solution. Instead of a flat feature matrix, I represent the network as a GRAPH — nodes are hosts, edges are connections. I then use a Graph Neural Network encoder that processes this graph. The key insight is that GNN weights are shared across all nodes and all graph sizes, so a policy trained on 5 hosts can be applied to 10 hosts using the exact same weights, no changes.

Think of it like this: a human security expert who has learned how to attack 5-machine networks doesn't need to go back to school when faced with a 10-machine network. They apply the same attack principles. GNN-DynPen does the same thing."

Time: ~90 seconds

SLIDE 3 — Literature Review

What to say:

"Let me briefly walk through the six key papers that directly influenced this project.

Starting top-left: NASim by Schwartz and Kurniawati in 2019 — this is the foundational work that built the simulator I use and proved that basic DQN can find attack paths in small static networks.

Below that, Hu et al. in 2020 extended this to work with attack tree structures. Still static.

Then Tran et al. in 2021 introduced hierarchical agents — a high-level planner and a low-level executor — which helped handle more complex scenarios.

On the right side: Chen et al.'s GAIL-PT in 2023 used expert demonstrations to speed up learning. Ghanem and Chen also in 2023 scaled hierarchical RL to larger networks.

And most importantly at the bottom right: GraphSAGE by Hamilton et al. and Graph Attention Networks by Veličković et al. — these are the two GNN architectures I considered. I chose GraphSAGE because it's inductive — it learns aggregation functions, not fixed node embeddings — which means it handles new graphs it's never seen before.

The key observation across all these papers: none of them except DynPen handle dynamic networks. And DynPen, while great at adapting, still uses a fixed-size input. That's the gap I address."

Time: ~90 seconds

Here's a clear summary and explanation of all six papers shown in the image:

1. DynPen — Li et al., IEEE TIFS 2024

DynPen proposes an automated reinforcement learning-based penetration testing scheme specifically designed for **dynamic network scenarios**. Prior RL-based tools mostly worked in static environments. DynPen addresses the challenge of networks that change over time. It captures observed changes in the environment to help the agent make better decisions based on historical experiences, and demonstrates significantly faster convergence speed while maintaining adaptability in dynamic settings. It serves as a **direct baseline** for newer work in this space.

2. GAT — Veličković et al., ICLR 2018

GAT (Graph Attention Network) introduced a novel neural network architecture for graph-structured data that uses masked **self-attentional layers**. By stacking layers where nodes attend over their neighbors' features, the model can assign different weights to different neighboring nodes — without expensive matrix operations or needing to know the graph structure in advance. GAT models achieved state-of-the-art results on four established graph benchmarks including citation networks and a protein-protein interaction dataset. It is described in the image as a more **expressive alternative to GraphSAGE** due to its attention mechanism.

3. NASim — Schwartz & Kurniawati, 2019

NASim (Network Attack Simulator) is a lightweight, high-level network attack simulator written in Python, designed for rapid testing of autonomous pen-testing agents using reinforcement learning and planning. It captures salient features of network pen-testing such as large and changing state/action spaces, partial observability, and varied network topology, and is modelled after the OpenAI Gym interface. It became one of the foundational environments for RL-based penetration testing research.

4. DeepExploit — Isozaki et al., 2018

DeepExploit is a fully automated penetration testing tool linked with Metasploit. It identifies the status of all open ports on a target server and executes exploits using machine learning. It uses A3C deep reinforcement learning, receives server information (OS type, product name, version) as inputs, and autonomously learns how to perform exploits — executing at pinpoint with as few as one attempt. Its key limitation, as noted in the image, is a **fixed action space and single topology**, which highlights a scalability gap compared to more recent approaches.

5. GraphSAGE — Hamilton et al., NeurIPS 2017

GraphSAGE is a general **inductive** framework that generates node embeddings by sampling and aggregating features from a node's local neighborhood. Unlike prior transductive methods, it doesn't need all nodes present during training — meaning it **generalizes to previously unseen nodes and graphs natively**. It demonstrated strong performance on inductive node-classification tasks including evolving citation graphs, Reddit posts, and protein-protein interaction graphs that were completely unseen during training.

6. CyberBattleSim — Microsoft Research, 2021

CyberBattleSim is a research platform for studying automated agents in a simulated abstract enterprise network. It uses an OpenAI Gym interface for training RL agents, and the environment is parameterized by a fixed network topology and a set of vulnerabilities agents can exploit to move laterally. It focuses specifically on the **post-breach lateral movement** stage of cyberattacks — the attacker's goal is to take ownership of network nodes while a defender agent monitors and tries to contain the attack. Its key contribution for the project shown in the image is its representation of networks as **heterogeneous-host graphs**, motivating graph-based state representations in RL-based pentesting.

SLIDE 4 — Research Gap

What to say:

"This slide captures the precise gap in one sentence, which I've highlighted in yellow here.

DynPen knows WHEN to adapt — it detects topology changes, backtracks to a valid state, and replans. That's impressive and useful. But it does NOT know HOW to encode the network state so that adaptation is cheap and transfer is possible.

There are three specific technical limitations I identified.

First — Fixed Input Size. DynPen's neural network has a hardcoded input dimension equal to $N_{\text{max}} \times F$, where N_{max} is the maximum host count set at training time. If you trained on 5 hosts and the real network has 6, the architecture breaks. You need to redesign and retrain.

Second — Quadratic Action Space. DynPen enumerates all possible (source host, destination host, exploit type) triples. If you have N hosts and V vulnerability types, that's $N^2 \times V$ actions. With 10 hosts and 5 exploit types, that's 500 actions. With 20 hosts it's 2000. The action space explodes, making learning unstable in larger networks.

Third — No Transfer Learning. Every new topology, every new network size means a completely new agent. There's no way to reuse the knowledge from previous training.

GNN-DynPen addresses all three. The GNN encoder is size-independent — same weights any N . The action space is linear — we score (target node, exploit type) pairs, which is $N \times K$, not $N^2 \times V$. And the trained weights transfer zero-shot to new topologies.

I want to emphasise: this isn't just an engineering improvement — it's a fundamentally different architectural philosophy."

Time: ~2 minutes

SLIDE 5 — Our Contributions

What to say:

"So what exactly did I do? Let me walk through the five concrete contributions.

Contribution 1 — GNN State Encoder. I replaced the flat host matrix with a 2-layer GraphSAGE network. Given a graph with N hosts, it outputs per-node embeddings of dimension 64, plus a global graph embedding also of dimension 64, using exactly the same weights regardless of N .

Contribution 2 — Variable Action Space. Instead of pre-enumerating all action triples, I score each (target node, exploit type) pair using the Q-value formula shown here: MLP of the graph embedding concatenated with the target node's embedding and the exploit type's embedding. This is $O(N \text{ times } K)$ — linear in network size.

Contribution 3 — Zero-shot Transfer. The trained GNN policy works on networks it has never been trained on, without any fine-tuning. I demonstrated this on 3-host, 8-host, and 10-host networks when training was only on 5-host networks.

Contribution 4 — Online Fine-tuning. After a topology change, the GNN can recover performance with only a short fine-tuning session of a few hundred steps, because the weights are already close to valid — unlike the baseline which needs to retrain from scratch.

Contribution 5 — Backtracking Integration. I integrated DynPen's three adaptation modules — trajectory logger, monitoring module, and backtracking module — on top of the GNN agent, and showed in RQ4 that they provide additional gains on top of the GNN alone."

Time: ~2 minutes

SLIDE 6 — System Architecture

What to say:

"This slide shows the full system pipeline. Let me walk through it left to right, top to bottom.

At the top, the NASim environment. At each timestep, it returns a graph observation — a set of host nodes with feature vectors and edges representing network connections, plus a reward signal.

That graph goes into the GNN Encoder — the GraphSAGE network I just described. It outputs node embeddings for each host and a graph-level embedding summarising the whole network.

Those embeddings go into the Policy Network — the Q-value scoring MLP. For each possible action (target host, exploit type), it computes a Q-value. The action with the highest Q-value is selected using epsilon-greedy.

The selected action goes to the Action Executor, which sends it to NASim's step function. NASim returns the next observation and reward.

That feeds back in a loop — standard RL training.

At the bottom, I've shown the three DynPen Adaptation Modules used in RQ4. The Trajectory Logger records every (state, action) pair. The Topology Monitor periodically checks if the network has changed. The Backtrack Module, when a change is detected, walks backward through the logged trajectory to find the most recent step that's still valid in the new topology, and resumes from there.

These three modules sit on top of the core agent — they don't change the GNN architecture, they just coordinate how the agent responds to changes."

Time: ~2 minutes

SLIDE 7 — Methodology

What to say:

"This slide covers the formal technical details.

On the left is my MDP formulation. The state is a dynamic graph G_t — the graph can change at each timestep. The actions are variable-length sets of (node, template) pairs. The reward is sparse and shaped — you get a big positive reward for compromising the goal host, a small negative per step to encourage efficiency, and a large negative if you trigger a honeypot. The transition is stochastic — exploits don't always succeed. And gamma of 0.99 means the agent values future rewards almost as much as immediate ones.

On the right are the key hyperparameters. I used a 2-layer GraphSAGE with embed dimension 64. Both the GNN and baseline use the same learning rate of $1e-3$ with Adam. The epsilon decay goes from 1 to 0.05 over 50,000 steps. I ran 5 independent seeds for each experiment and report mean plus or minus standard deviation.

One important note on experimental design: I ran both the GNN agent and the baseline with identical training budgets, optimisers, and epsilon schedules. This ensures any performance difference comes from the architecture, not from one agent having more training."

Time: ~90 seconds

SLIDE 8 — Experimental Setup & Research Questions

What to say:

"I organised my project around four research questions, each with a clear, testable criterion.

RQ1 asks: can the GNN learn as well as the baseline on the training topology? The criterion is GNN reward must be at least 90% of baseline reward. This checks that switching to GNN doesn't hurt performance on the simple case.

RQ2 asks: can the GNN generalise zero-shot to networks it was never trained on? The criterion is achieving more than 50% of baseline performance on at least 2 unseen topology sizes. This tests the core scalability claim.

RQ3 asks: after a topology change at runtime, does the GNN recover faster? The criterion is fewer episodes to recovery in the majority of change regimes. This tests the dynamic adaptation capability.

RQ4 asks: does adding DynPen's backtracking on top of the GNN provide extra benefit? The criterion is that fine-tuning plus backtracking outperforms fine-tuning alone. This tests whether the two mechanisms are complementary.

All four criteria were met, as I'll show in the next slides."

Time: ~90 seconds

SLIDE 9 — Results RQ1

What to say:

"Let's look at the results, starting with RQ1 — convergence on the fixed training topology.

On the left is the learning curve. The red line is the baseline DQN, the blue is GNN-DynPen. Both are trained on the same 5-host network for 100,000 steps. The shaded regions show standard deviation across 5 seeds.

A few things to notice. First, the GNN actually converges slightly FASTER than the baseline — it reaches 80% of its final performance at around step 35,000, while the baseline takes until step 45,000. This is because the GraphSAGE encoder provides richer state representations through neighbourhood aggregation — information about neighbouring hosts flows into each node's embedding, giving the agent a better picture of the network at each step.

On the right, the final performance bar chart. Baseline DQN reaches a mean return of 18.07. GNN-DynPen reaches 16.53. The ratio is 0.915, which is above the 0.90 criterion. So RQ1 is satisfied.

I should note: the GNN's advantage on the training topology is modest — both agents have enough capacity for 5 hosts. The real differentiation comes in the next three RQs."

Time: ~90 seconds

SLIDE 10 — Results RQ2

What to say:

"RQ2 is where the architectural difference becomes dramatic.

The grouped bar chart shows three conditions for each topology size: baseline retrained from scratch on that topology in red, GNN zero-shot in blue — meaning frozen weights from the 5-host training, no fine-tuning at all — and GNN retrained on that topology in green.

On the 3-host network: GNN zero-shot achieves 91% of the retrained baseline. On the 8-host network: 64%. On the 10-host network: 54%.

The critical observation: the red bars are not the GNN's competition — they represent re-training the baseline from scratch on each topology. Even without any retraining at all, the GNN gets 64% and 54% of a freshly trained specialist agent.

The baseline DQN bars are absent for 8-host and 10-host because those networks simply cannot be evaluated — the input layer expects 60 features from a 5-host network, but these have 96 and 120 features respectively. It's architecturally impossible without full redesign and retraining.

Both the 8-host and 10-host results are above the 50% criterion, so RQ2 is satisfied.

The performance drop with increasing N is expected and well-understood. The GNN uses 2-hop context — it can see 2 steps away in the graph. On larger, sparser networks, 2 hops may not be enough to reason about the full attack path. Training on mixed topology distributions or using 3 GNN layers would close this gap — that's in the future work."

Time: ~2 minutes

SLIDE 11 — Results RQ3

What to say:

"RQ3 tests what happens when the network CHANGES while the agent is already deployed.

I injected five types of topology changes and measured how many episodes it took to recover to 90% of pre-change performance — the Episodes-to-Recovery metric on the Y axis. Lower is better.

The red bars are the baseline, blue is GNN-DynPen.

For property change — a vulnerability is patched on one host — GNN recovers in about 14 episodes, the baseline in about 28. That's a 50% speedup.

For edge add and edge remove — connectivity changes — GNN recovers in 23 and 20 episodes respectively, versus 41 and 39 for the baseline. About 45-48% faster.

For honeypot injection — a host is secretly replaced with a honeypot that gives penalty rewards — GNN recovers in 31 episodes versus 47 for baseline. 33% faster. This is the hardest regime because the penalty is deceptive.

For the all-random regime — a mixture of all change types — GNN recovers 40% faster.

Why is GNN better here? The baseline's Q-values are indexed to fixed node positions — when node 3 becomes a honeypot, the agent's stored associations break. The GNN re-computes embeddings from the CURRENT graph structure at every step — it naturally adapts because the weights encode structural relationships, not node identities.

On average, GNN-DynPen recovers 43% faster — RQ3 criterion is satisfied."

Time: ~2 minutes

SLIDE 12 — Results RQ4

What to say:

"RQ4 is an ablation study — it tests which parts of the system actually contribute.

I compared three conditions under the all-random dynamic regime.

Condition A — Frozen policy, no fine-tuning, no backtracking. This is what happens if you just deploy the trained GNN and make no adjustments after a topology change. Post-change return drops to 4.1 — essentially the agent is failing almost every episode. This shows that fine-tuning IS necessary.

Condition B — Fine-tuning only, no backtracking. The GNN does brief online fine-tuning after a change but does NOT use the trajectory backtracking mechanism. Return recovers to 11.8 — much better than frozen. Recovery takes about 24 episodes.

Condition C — Fine-tuning plus backtracking — the full GNN-DynPen system. Return is 13.6 — an additional +15% over fine-tuning alone. Recovery time drops to 17.8 episodes — 26% faster.

Why does backtracking help on top of fine-tuning? Because fine-tuning alone means the agent keeps trying to execute an attack plan that is now invalid — it may be heading for a node that has been removed. Backtracking finds the last valid step in the attack trajectory and resumes from there, avoiding wasted exploration of now-invalid paths.

This confirms the two mechanisms solve DIFFERENT problems and are complementary: GNN handles state representation and size invariance, backtracking handles temporal coherence of the attack plan. RQ4 is satisfied."

Time: ~2 minutes

SLIDE 13 — Comparison with DynPen

What to say:

"Let me put everything in perspective by directly comparing GNN-DynPen with the DynPen baseline.

The table summarises the six key dimensions.

State representation: DynPen uses a flat host-feature matrix, GNN-DynPen uses a proper graph with nodes and edges.

Policy input size: DynPen is fixed — if you train on 5 hosts you can only use it on 5 hosts. GNN-DynPen is variable — any number of hosts, same weights.

Action space: DynPen is $O(N^2 \times V)$ — it grows quadratically. GNN-DynPen is $O(N \times K)$ — linear. Much more scalable.

New topology handling: DynPen requires full retrain. GNN-DynPen handles it zero-shot.

The four summary statistics below the table tell the story: 91.5% of baseline return with no loss of capability, generalisation to 3 out of 3 unseen topology sizes, 43% faster recovery, and +15% return with backtracking.

The key message for your teacher: DynPen solved the ADAPTATION problem — when to react to changes. GNN-DynPen solves the REPRESENTATION problem — how to encode state so the agent can handle different networks without retraining. These are orthogonal contributions and together they form a more complete solution."

Time: ~2 minutes

SLIDE 14 — Future Work

What to say:

"Let me briefly outline six directions for extending this work.

Top row: Larger topologies — I tested up to 10 hosts. Real enterprise networks have hundreds. Testing on 20-50 host NASim scenarios is the next logical step.

PPO / Actor-Critic — I used DQN which is off-policy and efficient but can be unstable in complex action spaces. PPO with a GNN actor is a promising alternative worth comparing.

Multi-agent penetration testing — add a second RL agent playing the defender, patching vulnerabilities in response to the attacker. Training both simultaneously as a competitive game would produce more robust attack policies and more realistic threat modelling.

Bottom row: Real vulnerability data — my simulator uses synthetic vulnerability categories. Using real CVE embeddings from the National Vulnerability Database would make experiments much more relevant to practitioners.

GNN Explainability — apply GNNExplainer to understand which edges and node features drive the agent's decisions. This matters for security auditors who need to understand why an attack path was chosen.

Blue-team adversary — a fully competitive attacker-defender setting modelled as a two-player zero-sum game, which is the most realistic and academically rich extension."

Time: ~90 seconds

SLIDE 15 — Summary

What to say:

"Let me close with a summary.

The problem: DynPen, the state-of-the-art adaptive PT agent, has a flat-matrix state representation that is fundamentally tied to one specific topology size. It cannot transfer.

My approach: Replace the flat matrix with a GraphSAGE GNN encoder — permutation-invariant, size-invariant, same weights work on any network.

Results across all four research questions:

RQ1: GNN converges to 91.5% of baseline on the training topology — no performance loss. RQ2: Zero-shot transfer to 3, 8, and 10-host networks the agent was never trained on. RQ3: 43% faster recovery after runtime topology changes on average. RQ4: Adding DynPen's backtracking on top of GNN gives a further +15% reward and 26% faster recovery.

The central practical implication: a single trained GNN-DynPen policy can be deployed against networks it has never seen. DynPen needed one agent per topology. GNN-DynPen needs one agent for all topologies.

Thank you. I'm happy to take questions."

Time: ~90 seconds

ANTICIPATED QUESTIONS AND ANSWERS

Q: Why GraphSAGE and not GAT (Graph Attention Network)? A: I evaluated both. GraphSAGE was chosen because mean aggregation is more stable during early training when GNN representations are noisy — the attention weights in GAT can overfit to small topology artifacts. GAT is listed as future work for more expressive aggregation.

Q: Isn't your simulator custom, not the real DynPen's NASim? How can you compare? A: Yes, this is acknowledged as a limitation in the paper. I built a NASim-compatible environment that follows the same state/action/reward formulation. Direct numerical comparison with published DynPen numbers requires the exact same simulator — that validation is future work.

Q: What about adversarial defenders? Your experiments have no active blue team. A: Correct — I model passive topology changes (patches, host additions), not an active defender. Multi-agent competitive training is the highest-priority future work item.

Q: Why Double DQN and not PPO or SAC? A: DQN's off-policy replay buffer reuses transitions efficiently, which is critical given the graph-by-graph processing overhead (each transition needs its graph structure stored). PPO would require on-policy rollouts which are

wasteful when each forward pass is expensive. SAC would require continuous action spaces — a redesign. DQN was the right fit for this initial implementation.

Q: How does the backtracking module know the topology has changed? A: The monitoring module computes a degree-sequence hash of the current graph at each check interval. If the hash differs from the stored snapshot, a change is flagged. This is cheap — it only requires reading adjacency, not running exploits.

Q: What is your biggest limitation? A: Scale. I tested up to $N=10$ hosts. Real enterprise networks have hundreds to thousands. The GNN architecture supports arbitrary N , but the training distribution ($N=5$) biases learned representations. Training on a curriculum of mixed topology sizes is needed before deployment on real networks.